

Débitos Recurrentes

Prueba Técnica — Analítico III

Jhon Fredy Correa Gomez

Economista · Científico de Datos

Bancolombia · Gerencia de Inteligencia de Originación y Cobranza · 2026

Agenda

#	Tema	Tiempo
1	Contexto de negocio y objetivo	1 min
2	Arquitectura de datos y pipeline	3 min
3	Análisis exploratorio — hallazgos clave	3 min
4	Feature engineering y selección	2 min
5	Modelos y resultados (AUC / KS)	3 min
6	Diagnóstico — AUC = 1.0	2 min
7	Dashboard Metabase	2 min
8	Operacionalización con Airflow	2 min
9	Conclusiones y próximos pasos	2 min

1. Contexto

Entendimiento del problema de negocio

Problema de negocio

¿Qué se quiere resolver?

Identificar obligaciones en mora temprana (1–30 días) con **alta probabilidad de pagar exclusivamente por débito recurrente**, para reducir costos de cobranza y mejorar la eficiencia operativa.

Definición de la variable respuesta

Clase	Condición	Registros	%
<code>var_rta = 1</code>	Pagos únicamente por débito Y recurrencia ≥ 40 %	36,835	78.8 %
<code>var_rta = 0</code>	Pago por otro canal O sin patrón de pago	9,901	21.2 %

Condición doble simultánea: exclusividad de canal + frecuencia mínima.

Un cliente con 60 % débito / 40 % otro canal → **clase 0**.

Desbalance **3.7:1** → estratificación + `class_weight` en todos los modelos.

Segmentación operativa de cobranza

Segmento	Criterio	Acción
A — Automatizar	var_rta=1 y mora ≤ 10 días	Sin gestión humana
B — Monitorear	var_rta=1 y mora > 10 días	Seguimiento ligero
C — Cobranza suave	var_rta=0 y mora ≤ 15 días	Contacto preventivo
D — Cobranza intensiva	var_rta=0 y mora > 15 días	Gestión activa

Impacto directo: concentrar recursos humanos solo en segmentos C y D.

2. Arquitectura

Diseño de datos y pipeline escalable

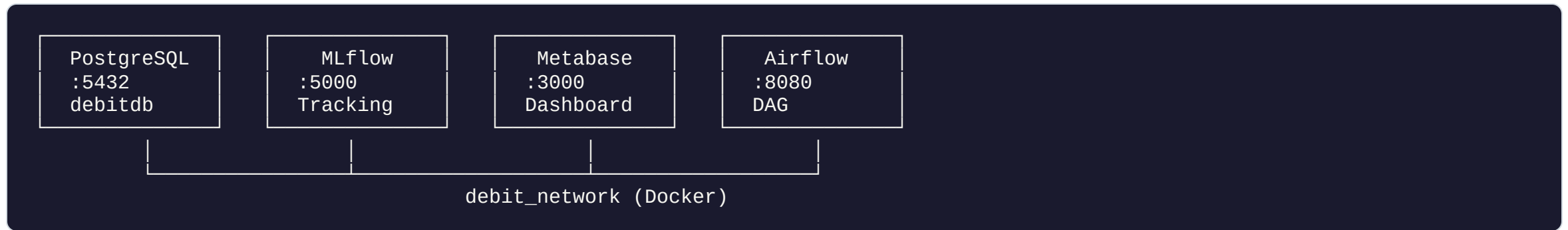
Fuentes de datos — 6 archivos CSV

Fuente	Filas	Cols	Contenido
clientes	46,736	4	Base maestra con var_rta
pagos (tanque)	47,194	67	Pagos por canal: débito, físico, virtual
gestiones	47,194	83	Gestiones de cobranza (acuerdos, RPC)
excedentes	47,194	35	Pagos sobre cuota y % de excedente
moras	47,194	12	Días de mora por ventana temporal
canales	10,801	4,258	Transacciones granulares por canal

Clave primaria compuesta: (num_doc, obl17, f_analisis)

Cobertura de canales: 20.7 % del universo → LEFT JOIN + imputar 0

Arquitectura de servicios



- **ORM:** SQLAlchemy → 6 tablas + 9 vistas `v_debit_*`
- **Imágenes:** `debit-ml:latest` · `debit-airflow:latest`
- **Migraciones:** Alembic (versionadas)
- **Artefactos:** `data/artifacts/` — parquets + PKL + `feature_cols.json`

Pipeline — flujo de datos

```
CSV datalake/
  |
  ▼
T1: loader.py          → PostgreSQL (debitdb)
  |
  ▼
T2: data_preparation.py → analytical_model.parquet (JOIN 6 fuentes)
  |
  ▼
T3: feature_engineering.py → train / test / oot .parquet (~240 features)
  |
  ▼
T3.5: feature_selection.py → feature_cols.json (varianza → ANOVA → ElasticNet)
  |
  ▼
T4: train_debit_classifier.py → PKL + MLflow (XGBoost, GBM, LogReg)
  |
  ▼
T5: metabase_provisioning.py → Dashboard id=4
```

Particionamiento Train / Test / OOT

Partición	Rango temporal	Periodos	Filas
Train	2024-07 → 2025-02	8	22,291
Test	2025-03 → 2025-07	5	13,712
OOT	2025-08 → 2025-11	4	10,733

Hallazgo clave: 97.9 % de obligaciones aparece en un único `f_analisis` — el dataset es casi **transversal**, no longitudinal. El split segmenta **cohortes distintas**, no evolución de la misma obligación.

3. EDA

Hallazgos empíricos clave

Hallazgo H1 — Clase 0 = sin actividad de pago

El 100 % de observaciones clase-0 tienen `total_pago = 0` en TODAS las ventanas y canales del tanque.

- `var_rta = 0` no significa "pagó por otro canal"
- Significa "no hay registro de pago" en el período analizado
- Implicación para el modelo: las features de débito son casi deterministas

```
Clase 0: avg_pago_debito_3m = 0 ✓  
         avg_pago_fisico_3m = 0 ✓  
         avg_pago_virtual_3m = 0 ✓  
         avg_pago_otros_3m = 0  ✓
```

Otros hallazgos relevantes

ID	Hallazgo
H2	14.9 % de clase-1 tiene pagos físico/virtual en ventana 3m. <code>var_rta</code> se calcula sobre <code>f_analisis</code> ; el tanque promedia los 3m previos.
H3	<code>rec_e_v</code> no es señal útil: solo 239 obs activas (< 0.1 %). El tanque es la fuente correcta.
H4	97.9 % de obligaciones en un único período. Split temporal = segmentación de cohortes.

Reconstrucción de `var_rta` desde el tanque

Estrategia	Concordancia	FP	FN
E1: avg_débito_3m > 0 + exclusividad	65.70 %	0	16,032
E2: prop_débito_3m ≥ 40 % + exclusividad	65.70 %	0	16,032
E3: avg_débito_12m > 0 + exclusividad	55.82 %	0	20,650
E4: min_débito_3m > 0 + exclusividad	65.38 %	0	16,181

Cero Falsos Positivos en todas las estrategias de ventana 3m.

La ventana de 3 meses es más limpia que la de 12m.

La baja concordancia refleja que `var_rta` usa una ventana puntual distinta al promedio del tanque.

4. Features

Ingeniería y selección de variables

Feature engineering — grupos de variables

Grupo	Fuente	Ejemplos	Cols
Pagos	tanque	avg/min/max/std_pago_[canal]_[3m...12m]	67
Derivadas	tanque	prop_debito_3m , canal_unico_debito_3m , debito_exclusivo_40pct_3m	21
Gestiones	gestiones	avg_cant_gestiones_3m , avg_rpc_3m	48
Moras	moras	avg_mora_3m , max_mora_6m	9
Excedentes	excedentes	avg_porc_pago_3m , avg_excedente_pago_6m	9
Canales	canales	trx_mnt_total , trx_cnt_total (solo 3 resumen)	4

Total candidatas: ~240 features

Canales — decisión de features (S10)

El archivo `canales` tiene **4,258 columnas**. Se descartaron 4,252 granulares; solo se retuvieron 3 de resumen.

Motivo	Detalle
Sparsity	~77 % de las cols granulares son cero → eliminadas en varianza de todas formas
Cobertura	Solo 20.7 % del universo tiene datos de canales (9,460 / 45,731 obligaciones)
Producción	4,252 columnas no son viables en inferencia, monitoreo ni mantenimiento
Duplicados irresolubles	88 claves con producto cartesiano en ETL fuente → excluidas; impacto -0.25 %

Columnas retenidas para el modelo:

`trx_mnt_total` · `trx_mnt_total_smmlv` · `trx_cnt_total`

Las 4,252 granulares se mantienen disponibles **solo para EDA y tablero Metabase**.

Selección supervisada — 3 etapas

```
240 candidatas
|
▼ Varianza / Sparsity (umbral 0.01)
166 features (-74)
|
▼ ANOVA F-test (p-valor < 0.05 ajustado)
132 features (-34)
|
▼ ElasticNet L1/L2 (l1_ratio=0.7, C=0.1)
116 features finales (-16)
```

`pago_fisico` y `pago_otros` **eliminados** por ANOVA → confirma H1.

Selección entrenada **solo en train** para evitar leakage.

5. Modelos

Entrenamiento, métricas y comparación

Stack de modelos

Modelo	Configuración	Ventaja
XGBoost	Optuna 5 trials · <code>scale_pos_weight</code> · early stopping	Robusto, interpretable vía SHAP
Gradient Boosting	HistGBM · Optuna 5 trials · <code>class_weight</code>	Más rápido, mejor CV
Logistic Regression	ElasticNet · <code>saga</code> · <code>class_weight</code>	Baseline lineal, alta interpretabilidad

Métrica de optimización: AUC-ROC (validación cruzada estratificada 5-fold)

Métricas de reporte: AUC + KS + Precision / Recall / F1

Resultados — Split aleatorio (producción)

Modelo	CV AUC	Train AUC	Test AUC	OOT AUC	KS Test
XGBoost	0.9587	0.9607	0.9594	0.9580	0.7300
Gradient Boosting	0.9616	0.9673	0.9620	0.9616	0.7379
Logistic Regression	0.9548	0.9547	0.9537	0.9530	0.7100

Estabilidad excelente: Train $\approx$ Test $\approx$ OOT — sin sobreajuste.

Top feature: `prop_debito_3m` (SHAP = 8.68).

Modelo de producción: **Gradient Boosting** (PKL en `data/artifacts/`).

6. Diagnóstico

¿Por qué $AUC = 1.0$ con split temporal?

Alerta 1 — AUC = 1.0 en Test/OOT (split temporal)

Síntoma:

```
Split temporal → CV Train: 0.84 | Test: 1.0000 | OOT: 1.0000  
Split aleatorio → CV Train: 0.96 | Test: 0.9594 | OOT: 0.9580
```

Causa: NO es leakage temporal clásico. Las ventanas de features son **previas** a `f_analisis`.

Causa real — interacción de dos hechos estructurales:

1. **H1:** Clase-0 = cero actividad de pago en todas las ventanas
2. **H4:** Dataset casi transversal — el split cronológico concentra clase-0 en cohortes específicas de Test/OOT

Las features de débito discriminan trivialmente a clase-0 → separación perfecta en esos conjuntos.

Decisión — Opción A (mantenida)

Opción	Features	AUC real	Uso en producción
A 	Todas (incluye débito)	~0.96	Cientes con historial de pagos
B	Sin features de débito	~0.80	Cientes nuevos / sin historial
C	Dos modelos (A + B)	mixto	Mayor cobertura, más complejidad

Decisión adoptada: Opción A. El historial de débito está disponible en producción para la cartera analizada. AUC real = **0.96** — excelente discriminación.

7. Dashboard

Tablero Metabase — análisis descriptivo

10 cards en Metabase — <http://localhost:3000/dashboard/4>

#	Card	Tipo	Fuente
1	Evolución % Débito Exclusivo por Período	Línea	<code>v_debit_kpi_period</code>
2	Volumen de Obligaciones por Período	Barras	<code>v_debit_kpi_period</code>
3	Segmentos de Cobranza — Distribución Global	Torta	<code>v_debit_risk_segment_summary</code>
4	Mix de Canales de Pago por Clase	Barras	<code>v_debit_payment_mix</code>
5	Efectividad de Gestiones por Clase	Barras	<code>v_debit_gestiones_profile</code>
6	Segmentos de Cobranza por Período	Barras	<code>v_debit_risk_segment_summary</code>
7	KPI Resumen — Último Período	Escalar	<code>v_debit_kpi_period</code>
8	Comparativa AUC y KS por Modelo	Tabla	<code>debit_model_metrics</code>
9	Estabilidad Train/Test/OOT por Modelo	Barras	<code>debit_model_metrics</code>
10	Top 20 Features por Modelo	Barras	<code>debit_model_features</code>

8. Operacionalización

DAG Airflow — ejecución programada

DAG `debit_ml_pipeline` — 7 tareas

```
gate_setup → setup_artifacts_dir
|
gate_load_data → load_data (CSV → PostgreSQL)
|
gate_build_model → build_analytical_model (JOIN 6 fuentes)
|
gate_build_features → build_features (~240 cols → parquets)
|
gate_select_features → select_features (116 features finales)
|
gate_train → train_model (XGBoost + GBM + LogReg + MLflow)
|
gate_metabase → provision_metabase (10 cards idempotente)
```

Cada **gate** es un `ShortCircuitOperator` configurable desde `.env` o al disparar el DAG.

Cada **tarea** corre como `DockerOperator` sobre `debit-network`.

Control de ejecución

Pipeline completo:

```
make airflow-trigger
```

Ejecución parcial desde UI (<http://localhost:8080>):

- Trigger DAG → formulario con 7 checkboxes → desmarcar pasos a omitir

Ejecución parcial desde CLI:

```
airflow dags trigger debit_ml_pipeline \  
--conf '{"run_load_data": false, "run_build_model": false}'
```

Calendarizable: [@monthly](#) por defecto — ajustable en el DAG.

9. Conclusiones

Resultados, supuestos y próximos pasos

Resultados principales

Logro	Detalle
Modelo en producción	Gradient Boosting · AUC = 0.96 · KS = 0.74
Estabilidad	Train ≈ Test ≈ OOT (sin degradación)
Pipeline end-to-end	CSV → PostgreSQL → Parquets → PKL → Metabase
Orquestación	Airflow DAG con gates configurables
Observabilidad	MLflow tracking + 10 cards Metabase
Diagnóstico transparente	Alerta AUC=1.0 identificada, explicada y resuelta

Supuestos clave adoptados

ID	Supuesto
S4	Canales ausentes → imputar 0 (no eliminar)
S9	<code>pago_debito</code> en tanque es señal directa del target — leakage justificado: disponible en producción
S10	Canales: solo 3 cols resumen (sparsity 77 % + cobertura 20.7 %)
S12	Clase 0 = sin patrón de pago, no "pagó por otro canal"
S13	Dataset casi transversal → split aleatorio es la evaluación más representativa

Próximos pasos

1. **Modelo sin historial (Opción B):** entrenar excluyendo features de débito → cobertura en clientes nuevos
2. **Umbral operativo:** ajustar punto de corte según costo de gestión vs. falso negativo
3. **Monitoreo de drift:** PSI mensual sobre distribución de `prop_debito_3m` y score
4. **Re-entrenamiento automático:** trigger en DAG si AUC OOT < 0.90
5. **ALERTA 3 pendiente:** aumentar `max_iter` en LogisticRegression para convergencia

¡Gracias!

Jhon Fredy Correa Gomez

Repositorio · Dashboard <http://localhost:3000/dashboard/4>

MLflow <http://localhost:5000> · Airflow <http://localhost:8080>

jonfredi12@gmail.com